

操作系统复习

为什么要虚拟内存？

核心背景与问题

内存管理的重要性：内存管理是操作系统最重要的部分之一。如果不懂内存管理，就无法理解操作系统的工作原理。

物理内存的局限性：

在早期的计算机中，程序直接在物理内存上运行。

多任务冲突：如果两个程序同时运行，它们可能会争夺同一块物理内存地址，导致数据覆盖或程序崩溃。

地址不隔离：一个程序如果错误地修改了另一个程序的内存，会导致系统不稳定。

虚拟内存的解决方案

核心概念：操作系统为每个进程提供一个独立的、连续的“假”内存空间，称为虚拟内存。

地址转换：

虚拟地址：程序运行时使用的是虚拟地址。

物理地址：数据实际存储在物理内存中的地址。

MMU（内存管理单元）：硬件设备，负责将虚拟地址转换为物理地址。

好处：

隔离性：每个进程都有自己独立的虚拟地址空间，互不干扰。

连续性：程序看到的内存是连续的，即使物理内存是碎片化的。

安全性：防止进程非法访问其他进程或操作系统的内存。

内存分段

分段机制：将虚拟内存分成不同的段（如代码段、数据段、堆、栈等）。

段表：操作系统维护一个段表，记录每个段的基地址和界限。

地址转换过程：

1. 程序生成虚拟地址（段号 + 段内偏移）。
2. MMU 查段表，找到该段在物理内存的基地址。
3. 基地址 + 段内偏移 = 物理地址。

缺点：

外部碎片：内存中会出现很多小的空闲块，难以利用。

内存交换效率低：分段大小不固定，进行内存交换（Swap）时比较麻烦。

内存分页

分页机制：为了解决分段的外部碎片问题，引入分页。

页：将虚拟内存和物理内存都切分成固定大小的块（通常为 4KB）。

页表：记录虚拟页号和物理页框号的映射关系。

地址转换：

虚拟地址分为：页号 + 页内偏移。

MMU 通过页表找到对应的物理页框号。

物理地址 = 物理页框号 + 页内偏移。

优点：消除了外部碎片，内存利用率高，便于内存交换（以页为单位）。

多级页表

问题：如果虚拟地址空间很大（如 32 位或 64 位），页表本身会非常巨大，占用大量内存。

解决方案：多级页表（如二级页表、三级页表）。

将页表分级存储，只加载当前需要的部分页表到内存。

通过一级一级地查找，最终找到物理页框。

TLB

问题：每次访问内存都要查页表（可能在内存中），导致访问速度变慢（需要访问两次内存：一次查页表，一次取数据）。

解决方案：TLB。

TLB 是 MMU 中的高速缓存，存储最近常用的页表项。

访问内存时，先查 TLB，如果命中（TLB Hit），直接得到物理地址，速度极快。

如果未命中（TLB Miss），再去查内存中的页表，并更新 TLB。

段页式内存管理

结合分段和分页：现代操作系统通常结合两者的优点。

分段：从逻辑上将程序分为不同的段（代码、数据等），提供保护和共享。

分页：在每个段内部进行分页，解决外部碎片问题。

Linux 的实现：Linux 主要采用分页机制，但在逻辑上也有分段的概念（但段基址通常为 0，实际上不起作用，主要是为了兼容 x86 架构）。

Linux 内存布局

32 位系统的虚拟地址空间布局（通常为 4GB）：

用户空间（0 ~ 3GB）：每个进程独立，存放代码、数据、堆、栈等。

内核空间（3GB ~ 4GB）：所有进程共享，存放内核代码和数据。

64 位系统：地址空间极大，用户空间和内核空间的划分比例不同（如 128TB 用户空间，128TB 内核空间）。

总结

虚拟内存的作用：

为每个进程提供独立的地址空间。

通过 MMU 进行地址转换。

利用分页解决碎片问题，利用 TLB 加速地址转换。

Linux 的内存管理：

主要基于分页。

虚拟地址空间分为用户空间和内核空间。

用户空间包含代码段、数据段、BSS 段、堆、栈等。

进程、线程、协程基础知识

我们编写的代码只是一个静态文件，通过编译后就会生成二进制的可执行文件，当执行这个可执行文件后，他就会被加载到内存中，接着CPU会执行程序中的每一条指令，这个运行中的程序，就被称为进程

进程、线程、协程的区别和联系

进程：

定义：资源分配和拥有的基本单位

切换情况：进程CPU环境（栈、寄存器、页表和文件句柄等）的保存以及新调度的进程CPU环境的设置

切换者：操作系统

切换过程：用户态->内核态->用户态

调用栈：内核栈

拥有资源：CPU资源、内存资源、文件资源和句柄

并发性：不同进程之间切换实现并发，各自占有CPU实现并行

系统开销：切换虚拟地址空间，切换内核栈和硬件上下文，CPU高速缓存失效、页表切换，开销很大

通信方式：进程间通信需要借助操作系统

线程

定义：程序执行的基本单位

切换情况：保存和设置程序计数器、少量寄存器和栈的内容

切换者：操作系统

切换过程：用户态->内核态->用户态

调用栈：内核栈

拥有资源：程序计数器、寄存器、栈和状态字

并发性：一个进程内部多个线程并发执行

系统开销：切换时只需保存和设置少量寄存器内容，因此开销较小

通信方式：线程间可以直接写进程数据段（如全局变量）来进行通信

协程

定义：用户态的轻量级线程，线程内部调度的基本单位

切换情况：先将寄存器上下文和栈保存，等切换回来的时候再进行恢复

切换者：用户

切换过程：用户态（没有陷入内核态）

调用栈：用户栈

拥有资源：拥有自己的寄存器上下文和栈

并发性：同一时间只能执行一个协程，而其他协程处于休眠状态，适合对任务进行分时处理

系统开销：直接操作栈则基本没有内核切换的开销，可以不加锁的访问全局变量，所以上下文切换非常快

通信方式：共享内存，消息队列

1、进程是资源分配的基本单位，运行一个可执行程序会创建一个或多个进程，进程就是运行起来的可执行程序

2、线程是资源调度的基本单位，也是程序执行的基本单位，是轻量级的进程，每个进程中都有唯一的主线程，且只能有一个，主线程和进程是相互依存的关系，主线程结束进程也会结束。

协程是用户态的轻量级线程，线程内部调度的基本单位

进程的状态

一个进程又三种基本状态：运行状态、阻塞状态、就绪状态

运行状态：该时刻进程占用 CPU；

就绪状态：可运行，由于其他进程处于运行状态而暂时停止运行；

阻塞状态：该进程正在等待某一事件发生（如等待输入/输出操作的完成）而暂时停止运行，这时，即使给它CPU控制权，它也无法运行；

此外，进程还有两个基本状态

创建状态：进程正在被创建时的状态；

结束状态：进程正在从系统中消失时的状态；

如果有大量进程阻塞，进程可能会占用物理内存空间，很显然我们不希望发生这种情况。

所以，在虚拟内存管理的操作系统中，通常会把阻塞状态的进程的物理内存空间换出到硬盘，等需要再次运行的时候，再从硬盘换入到物理内存。

那么，就需要一个新的状态，来描述进程没有占用实际的物理内存空间的情况，这个状态就是挂起状态。这跟阻塞状态是不一样，阻塞状态是等待某个事件的返回。

另外，挂起状态可以分为两种：

- 阻塞挂起状态：进程在外存（硬盘）并等待某个事件的出现；
- 就绪挂起状态：进程在外存（硬盘），但只要进入内存，即刻立刻运行；

导致进程挂起的原因不只是因为进程所使用的内存空间不在物理内存，还包括如下情况：

- 通过 `sleep` 让进程间歇性挂起，其工作原理是设置一个定时器，到期后唤醒进程。
- 用户希望挂起一个程序的执行，比如在 Linux 中用 `Ctrl+Z` 挂起进程；

进程控制结构

在操作系统中，是用进程控制块（PCB）数据结构来描述进程的

PCB包含的信息有：

进程描述信息：

- 进程标识符：标识各个进程，每个进程都有一个并且唯一的标识符；
- 用户标识符：进程归属的用户，用户标识符主要为共享和保护服务；

进程控制和管理信息：

- 进程当前状态，如 **new**、**ready**、**running**、**waiting** 或 **blocked** 等；
- 进程优先级：进程抢占 CPU 时的优先级；

资源分配清单：

- 有关内存地址空间或虚拟地址空间的信息，所打开文件的列表和所使用的 I/O 设备信息。

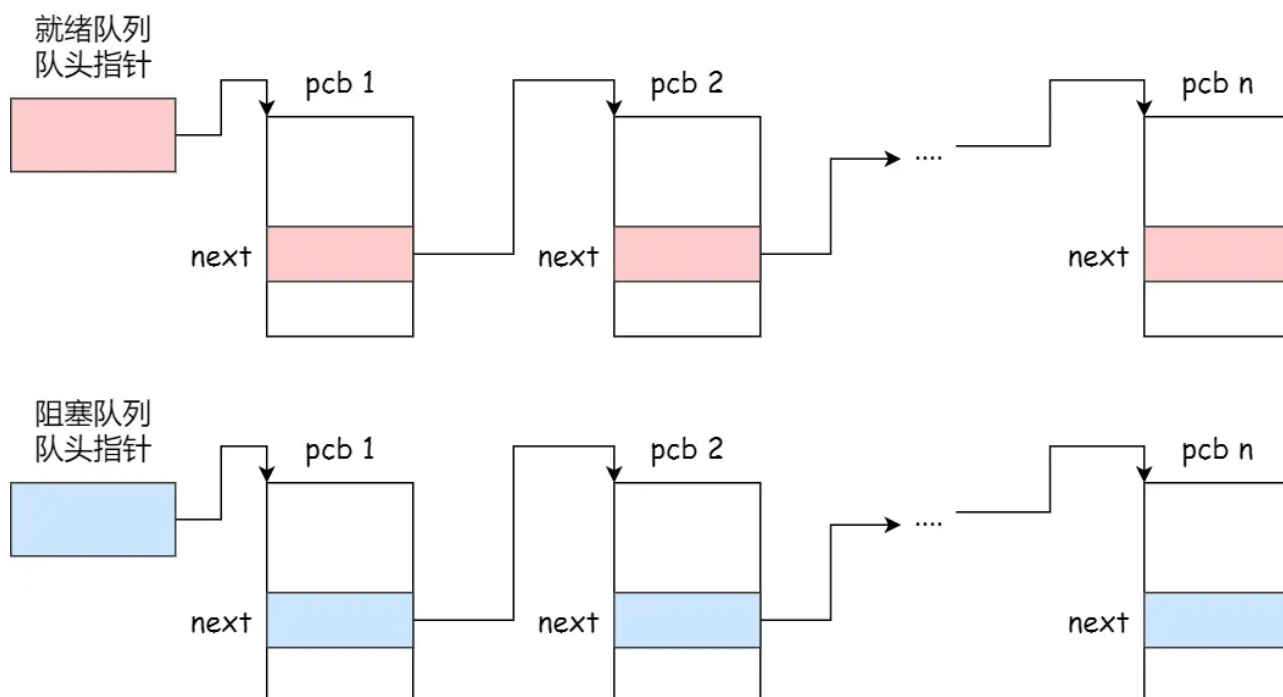
CPU 相关信息：

- CPU 中各个寄存器的值，当进程被切换时，CPU 的状态信息都会被保存在相应的 PCB 中，以便进程重新执行时，能从断点处继续执行。可见，PCB 包含信息还是比较多的。

PCB是如何组织的？

一般通过链表的方式进行组织，把具有相同状态的进程链在一起，组成各种队列

- 将所有处于就绪状态的进程链在一起，称为就绪队列；
- 把所有因等待某事件而处于等待状态的进程链在一起就组成各种阻塞队列；
- 另外，对于运行队列在单核 CPU 系统中则只有一个运行指针了，因为单核 CPU 在某个时间，只能运行一个程序。



进程的控制

进程控制是操作系统内核通过原语（Primitive）对进程生命周期进行管理的过程。它主要包括进程的创建、终止、阻塞和唤醒。

1. 创建进程

进程创建通常由一个已有进程（父进程）发起，创建一个新进程（子进程）。

1. 申请空白**PCB**：进程控制块（PCB）是进程存在的唯一标志。系统首先申请一个空白的PCB，并分配唯一的进程标识符（PID）。
2. 分配资源：为新进程分配运行所需的资源，如内存空间、I/O设备等。
3. 初始化**PCB**：填写进程标识、处理机状态（寄存器值）、进程控制通信（优先级、状态等）。
4. 插入就绪队列：将初始化好的PCB插入系统的就绪队列中，等待调度程序调度。

2. 终止进程

进程可以通过正常结束、异常出错或外界干预（如用户按Ctrl+C或执行kill命令）来终止。

1. 查找**PCB**：根据进程标识符找到要终止的进程PCB。
2. 终止运行：如果进程处于运行状态，立即终止其执行，并设置调度标志，以便调度程序重新调度。
3. 处理子孙进程：如果该进程有子孙进程，通常需要将它们终止（级联终止），或者将其托管给1号进程（init）收养。
4. 回收资源：释放该进程占用的所有资源（内存、文件等）归还给父进程或系统。
5. 撤销**PCB**：将该进程的PCB从所在队列或链表中移出，并撤销其PCB。

3. 阻塞进程

当进程等待某事件发生（如I/O操作完成、申请缓冲空间失败）时，它会主动调用阻塞原语把自己挂起。

1. 停止运行：找到要阻塞的进程PCB，如果正在运行，保护现场，将状态改为“阻塞状态”，停止运行。
2. 入队：将该PCB插入到等待该事件发生的阻塞队列中。

4. 唤醒进程

处于阻塞状态的进程无法自我唤醒，必须由其他进程（或操作系统）在事件发生时调用唤醒原语。

1. 出队：在阻塞队列中找到等待该事件的进程PCB，将其从队列中移出。
 2. 状态转换：将进程状态从“阻塞”改为“就绪”。
 3. 入就绪队列：将该PCB插入就绪队列，等待调度。
-

进程的上下文切换

上下文切换是操作系统实现多任务并发的核心机制。当CPU从一个进程切换到另一个进程时，必须保存当前进程的状态，并恢复下一个进程的状态。

- 什么是**CPU**上下文：
CPU上下文包括CPU寄存器和程序计数器（PC）。寄存器是CPU内部的高速存储单元，用于暂存指令和数据；PC记录当前指令的执行位置。这就像CPU的“口袋”，装着当前工作最紧要的东西。
- 切换过程：
 - a. 保存现场：将当前进程的CPU上下文（寄存器值、PC值）保存到该进程的PCB中。
 - b. 恢复现场：从下一个要运行的进程的PCB中读取之前保存的CPU上下文，加载到CPU寄存器和PC中。
 - c. 跳转执行：CPU根据新的PC值跳转到对应的指令位置继续执行。
- 切换内容：
除了CPU寄存器，上下文切换还涉及虚拟内存、栈、全局变量等用户空间资源，以及内核栈、硬件上下文等内核空间资源。
- 什么时候发生？
 - a. 时间片轮转：当前进程的时间片用完，系统强制进行调度。
 - b. 主动阻塞：进程因等待资源（如I/O）而主动放弃CPU。
 - c. 优先级抢占：有更高优先级的进程进入就绪队列，抢占当前进程的CPU。
 - d. 中断处理：硬件中断发生后，内核处理完中断可能需要切换到其他进程。

进程切换为什么比线程慢？

进程切换涉及虚拟地址空间的切换而线程不会。

因为每个进程都有自己的虚拟地址空间，而线程是共享所在进程的虚拟空间的，所以同一个进程中的线程进行线程切换时不涉及虚拟地址空间的转换。

把虚拟地址转换为物理地址需要查找页表，页表查找是一个很慢的过程（至少访问两次内存），因此通常使用**Cache**来缓存常用的地址映射，这样就可以加速页表查找，这个**cache**就是**TLB**（快表）

由于每个进程都有自己的虚拟地址空间，那么显然每个进程都有自己的页表，那么当进程切换后页表也要进行切换，页表切换后TLB就失效了。**cache**失效导致命中率降低，那么虚拟地址转换为物理地址就会变慢，表现出来的就是程序运行会变慢，而线程切换则不会导致TLB失效，因为线程无需切换地址空间，这也就是进程切换要比线程切换慢的原因。

一个进程可以创建多少线程，和什么有关？

这得分不同系统：

- 如果是32位系统，用户态的虚拟空间只有3G，如果创建线程时分配的栈空间是10M，那么一个进程最多只能创建300个左右的线程
- 如果是64位系统，用户态的虚拟空间大到有128T，理论上不会受到虚拟内存大小的限制，而会受系统的参数或性能限制

过多的线程将会导致大量的时间浪费在线程切换上，给程序运行效率带来负面影响，无用线程要及时摧毁

外中断和异常有什么区别？

外中断是指由CPU执行指令以外的事件引起，如I/O完成中断，表示设备输出/输入处理已经完毕，处理器能够发送下一个输入/输出请求。此外还有时钟中断、控制台中断等。

而异常是由CPU执行指令的内部事件引起，如非法操作码、地址越界、算法溢出等。

内存交换和覆盖有什么区别？

交换技术主要是在不同进程（或作业）之间进行，而覆盖则用于同一程序或进程中

虚拟技术是什么？

虚拟技术把一个物理实体转换为多个逻辑实体。

主要虚拟技术有两种：时（时间）分复用技术和空（空间）分复用技术

多进程与多线程：多个进程能在同一个处理器上并发执行使用了时分复用技术，让每个进程轮流占用处理器，每次只执行一小段时间片并快速切换

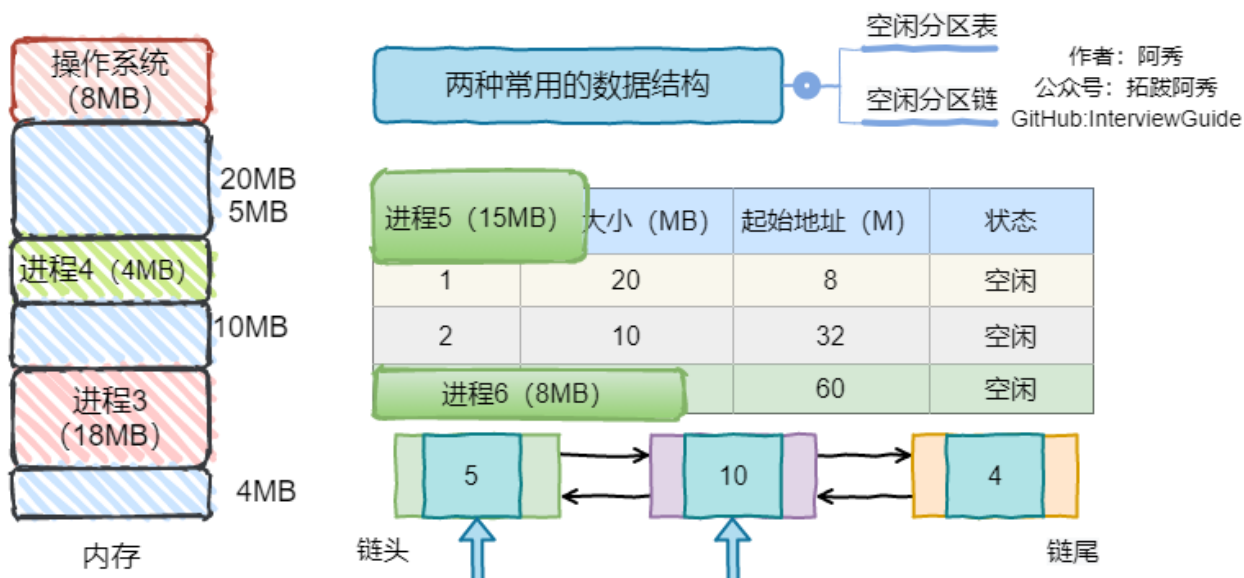
虚拟内存使用了空分复用技术，他将物理内存抽象为地址空间，每个进程都有各自的地址空间，地址空间的页被映射到物理内存，地址空间的页并不需要全部在物理内存中，当使用到一个没有在物理内存中的页时，执行页面置换算法，将该页置换到内存中。

动态分区分配算法有哪几种？

1.首次适应算法

算法思想：每次都从低地址开始查找，找到第一个能满足大小的空闲分区

如何实现：空闲分区以地址递增的次序排列。每次分配内存时顺序查找空闲分区链（或空闲分区表），找到大小能满足要求的第一个空闲分区。



2.最佳适应算法

算法思想：由于动态分区分配是一种连续分配方式，为各进程分配的空间必须是连续的一整片区域。因此为了保证当“大进程”到来时到来时能有连续的大片空间，可以尽可能多地留下大片的空闲区，即优先使用更小的空闲区

如何实现：空闲分区按容量递增次序链接。每次分配内存时顺序查找空闲分区链（或空闲分区表），找到大小能满足要求的第一个空闲分区。

什么是守护进程？

守护进程是指在后台运行的，没有控制终端与他相连的进程。他独立于控制终端，周期性地执行某种任务。

Linux地大多数服务器就是用守护进程地方式实现的，如web服务器进程http等。

创建守护进程要点

(1) 让程序在后台执行。

方法是调用`fork()`产生一个子进程然后使父进程退出

(2) 调用`setsid()`创建一个新对话期

守护进程需要摆脱父进程的影响，方法是调用`setsid()`使进程成为一个会话组长。`setsid()`调用成功后，进程成为新的会话组长和进程组长，进程组和控制终端脱离。

(3) 禁止进程重新打开控制终端

经过1和2，进程已经成为一个无终端的的会话组长，但是它可以重新申请打开一个终端。为了避免这种情况发生，可以通过使进程不再是会话组长来实现。再一次通过`fork()`创建新的子进程，使调用`fork`的进程退出

(4) 关闭不再需要的文件描述符

子进程从父进程继承打开的文件描述符。如不关闭，将会浪费系统资源，造成进程所在的文件系统无法卸下以及引起无法预料的错误。首先获得最高文件描述符值，然后用一个循环程序，关闭0到最高文件描述符值的所有文件描述符

(5) 将当前目录更改为根目录

(6) 子进程从父进程继承的文件创建屏蔽字可能会拒绝某些许可权。

为防止这一点，使用`unmask(0)`将屏蔽字清零

(7) 处理SIGCHLD信号。

对于服务器进程，在请求到来时往往生成子进程处理请求。如果子进程等待父进程捕获状态，则子进程将成为僵尸进程（zombie），从而占用系统资源。如果父进程等待子进程结束，将增加父进程的负担，影响服务器进程的并发性能。在Linux下可以简单地将SIGCHLD信号的操作设为SIG_IGN。这样，子进程结束时不会产生僵尸进程。

僵尸进程

多进程程序，父进程一般需要跟踪子进程的退出状态，当子进程退出，父进程在运行，子进程必须等到父进程捕获到了子进程的退出状态才真正结束。在子进程结束后，父进程读取状态前，此时子进程为僵尸进程。

设置僵尸进程的目的是维护子进程的信息，以便父进程在以后某个时候获取。这些信息至少包括进程ID，进程的终止状态，以及该进程使用的CPU时间。所以当终止子进程的父进程调用wait或waitpid时就可以得到这些信息。

但是子进程停止在僵尸态会占据内核资源，所以需要避免僵尸进程的产生或立即结束子进程的僵尸态。

1. 父进程调用wait/waitpid等函数等待子进程结束，如果尚无子进程退出wait会导致父进程阻塞。waitpid只会等待由pid参数指定的子进程，同时也是非阻塞，目标进程正常退出返回子进程PID，还没结束返回0。
 2. 在事件已经发生情况下执行非阻塞调用可以提高程序效率。对waitpid，最好在子进程退出后调用。使用SIGCHLD信号通知父进程，子进程结束。父进程中捕获信号，然后在信号处理函数中调用waitpid以彻底结束子进程
 3. 通过signal(SIGCHLD, SIG_IGN)通知内核对子进程的结束不关心，由内核回收。如果不想让父进程挂起，可以在父进程中加入一条语句：signal(SIGCHLD,SIG_IGN);表示父进程忽略SIGCHLD信号，该信号是子进程退出的时候向父进程发送的。
 4. 忽略SIGCHLD信号，这常用于并发服务器的性能的一个技巧因为并发服务器常常fork很多子进程，子进程终结之后需要服务器进程去wait清理资源。如果将此信号的处理方式设为忽略，可让内核把僵尸子进程转交给init进程去处理，省去了大量僵尸进程占用系统资源。
-

线程

线程是进程中的一个执行流，是CPU调度的基本单位。引入线程是为了减少并发执行时的时间和空间开销，提高系统的并发性能。

为什么使用线程

以视频播放器为例，如果采用单进程/单线程模型，程序必须串行执行：读取文件 -> 解码数据 -> 播放画面。一旦I/O操作（如读硬盘）耗时较长，整个界面就会卡顿。

- 多线程优势：
 - a. 提高响应速度：可以将I/O、解码、播放分配给不同的线程，互不干扰。
 - b. 资源共享：同一进程内的线程共享进程的内存空间（代码段、数据段、文件描述符），通信方便且开销小。
 - c. 经济高效：创建和切换线程的开销远小于进程。

线程与进程的比较

调度：线程是调度的基本单位，进程是资源分配的基本单位。

并发性：进程之间可以并发，进程内的多个线程也可以并发，大大提高了系统的并发度。

资源拥有：进程拥有系统资源，线程只拥有少量资源（寄存器、栈），但共享所属进程的资源。

独立性：进程之间相互独立，线程之间共享内存，因此线程间通信更简单，但同步互斥问题更复杂。

线程的实现方式

线程的实现主要分为用户级线程、内核级线程和组合方式。

1. 用户级线程

- 机制：线程的管理（创建、同步、调度）完全由应用程序在用户空间通过线程库完成，内核意识不到线程的存在。内核调度的单位仍然是进程。
- 优点：
 - 线程切换快，不需要切换到内核态。
 - 调度算法可以是特定于应用的。
- 缺点：
 - 阻塞问题：如果一个用户级线程执行系统调用被阻塞，整个进程（包括所有线程）都会被内核阻塞。
 - 多核利用率低：内核一次只分配一个CPU给进程，进程内的多个线程无法在多个CPU上并行运行。

2. 内核级线程

- 机制：线程的管理（创建、撤销、切换）全部由操作系统内核完成。内核维护线程控制块（TCB）。
- 优点：
 - 并行能力强：同一个进程的多个线程可以分配到多个CPU上并行执行。
 - 健壮性好：一个线程阻塞不会影响同进程的其他线程。
- 缺点：
 - 线程切换需要进出内核态，开销相对较大。

3. 组合方式（轻量级进程）

- 机制：结合了用户级和内核级的优点。用户级线程通过轻量级进程（LWP）连接到内核级线程。LWP是用户线程和内核线程之间的桥梁。
- 模型
 - **N:1** 模型：多个用户线程对应一个内核线程（纯用户级）。
 - **1:1** 模型：一个用户线程对应一个内核线程（纯内核级）。
 - **M:N** 模型：M个用户线程对应N个内核线程（组合式）。

怎么回收线程？有哪几种方式

- 等待线程技术： `int pthread_join(pthread_t tid, void** retval)`

主线程调用，等待子进程退出并回收其资源，类似于进程中wait/waitpid回收僵尸进程，调用pthread_join的线程会被阻塞

tid:创建线程时通过指针得到tid值。

retval:指向返回值的指针

- 结束线程： `void pthread_exit(void *retval);`

子线程执行，用来结束当前线程并通过retval传递返回值，该返回值可通过pthread_join获得

retval:指向返回值的指针

- 分离线程： `int pthread_detach(pthread_t tid);`

主线程、子线程均可调用。主线程中pthread_detach(tid)，子线程中pthread_detach(pthread_self())，调用后和主线程分离，子线程结束时自己立即回收资源

tid:同上

进程间的通信方式

管道

在linux中，有一个|的竖线符号，这就是一个管道，一般我们会在两个命令中间加一个“|”，代表将前一个命令的输出作为后一个命令的输入，从这功能描述，可以看出管道传输数据是单向的，如果想相互通信，需要两个管道。

I 这个管道没有名字，所以叫匿名管道，用完了就销毁

管道还有一个类型是命名管道，也被叫做FIFO，因为数据是先进先出的传输方式。

使用命名管道前，先需要通过**mkfifo**命令来创建，并且指定管道名字

```
mkfifo MyPipe
```

这样就创建了一个名为MyPipe的管道文件，然后向管道文件中写入数据

```
echo "hello" > MyPipe //将数据写入管道  
//卡住了
```

必须要有一个进程/线程将数据读走，写入端才能正常退出

```
cat < MyPipe //读取管道里的数据  
hello
```

可以看出管道是单向通信的，并且管道这种通信方式效率低，不适合进程间频繁地交换数据

原理：

匿名管道地创建，需要通过下面这个系统调用

```
int pipe(int fd[2])
```

这里表示创建一个匿名管道，并且返回两个描述符，一个是读端描述符fd[0]，一个是写端描述符fd[1]。注意：这个匿名管道是个特殊的文件，它只存在于内存中，不在文件系统中

所谓的管道，就是内核里面的一段缓存。从管道写入的数据是缓存在内核中的，另一端读取，也就是从内核中读取这段数据，并且管道传输的数据是无格式流而且大小受限。

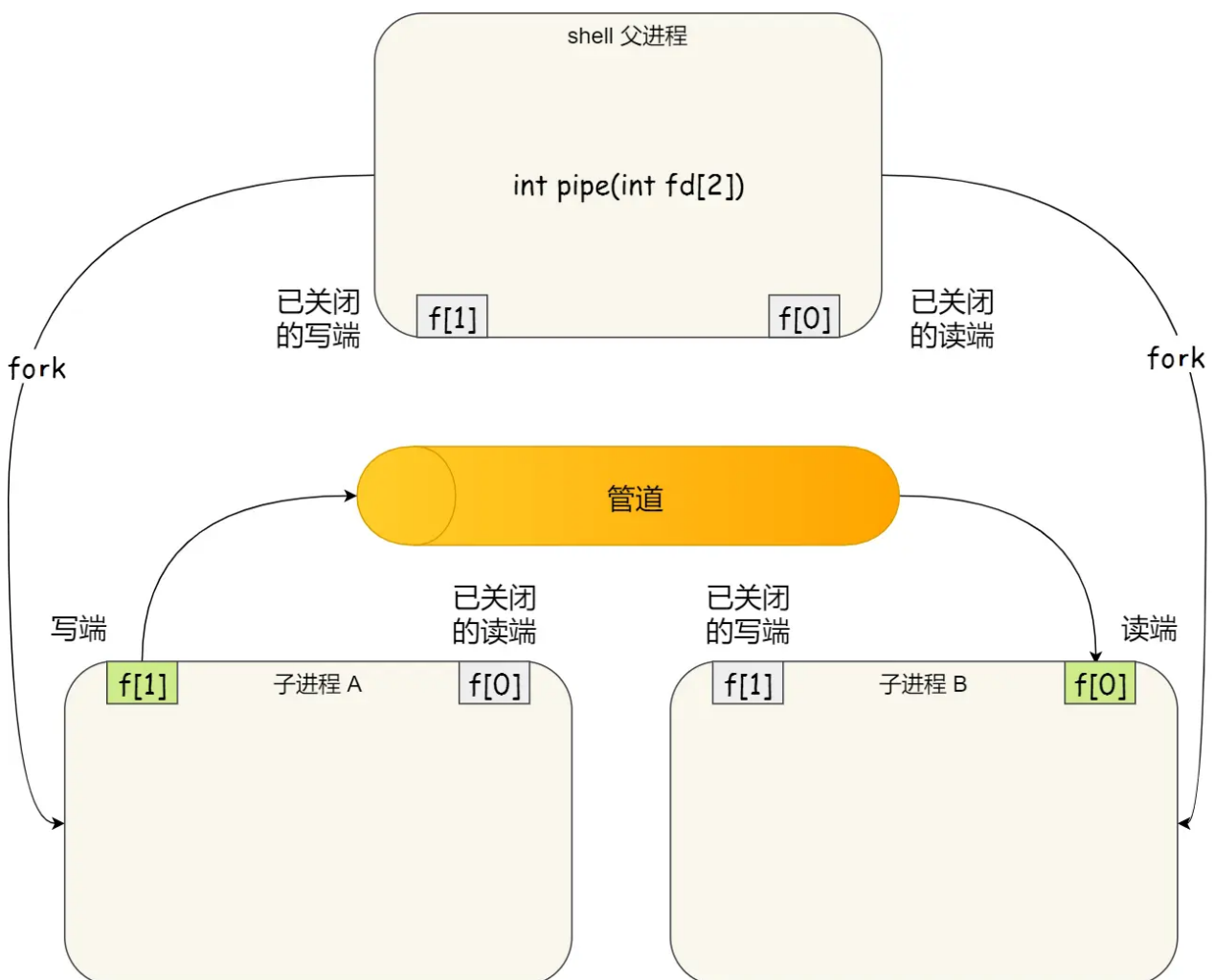
那么怎么进行进程间通信呢？

一般对于匿名管道来说，在一个进程中创建一个匿名管道，就会得到两个文件描述符，`fd[0]`和`fd[1]`，然后再创建一个子进程，子进程会继承父进程地两个文件描述符，所以一段写，一端发就可以了。

注意：我们会发现父子进程都能读或写管道，但是这样就可能出现差错，所以一般会关闭父进程的读（`fd[0]`）和子进程的写（`fd[1]`），这样就是父进程写子进程读，不会出现差错

但是如果要实现双向通信，那么还需要建立一个匿名管道，然后父进程负责读，子进程负责写

用管道符（`|`）输入命令时：



对于匿名管道，它的通信范围是父子关系的进程，也可以是亲兄弟之间的通信。因为匿名管道没有对应的文件，所以只能通过父进程`fork`子进程，将文件描述符传递给子进程。

另外,对于命名管道，它可以在不相关的进程间通信。因为命名管道，提前创建了一个类型为管道的设备文件，在进程里只要使用这个设备文件，就可以相互通信。

不管是匿名管道还是命名管道，进程写入的数据都缓存在内核中，另一个进程读取数据的时候也是从内核缓冲区中读走数据。

同时通信数据遵循先进先出原则，不支持lseek之类的文件定位操作。

消息队列

管道的通信方式是效率低的，因此管道不适合进程间频繁地交换数据。对于这个问题，消息队列的通信模式就可以解决。

比如，A 进程要给 B 进程发送消息，A 进程把数据放在对应的消息队列后就可以正常返回了，B 进程需要的时候再去读取数据就可以了。同理，B 进程要给 A 进程发送消息也是如此。

再来，消息队列是保存在内核中的消息链表，在发送数据时，会分成一个一个独立的数据单元，也就是消息体（数据块），消息体是用户自定义的数据类型，消息的发送方和接收方要约定好消息体的数据类型，所以每个消息体都是固定大小的存储块，不像管道是无格式的字节流数据。如果进程从消息队列中读取了消息体，内核就会把这个消息体删除。

消息队列生命周期随内核，如果没有释放消息队列或者没有关闭操作系统，消息队列会一直存在，而前面提到的匿名管道的生命周期，是随进程的创建而建立，随进程的结束而销毁。

消息这种模型，两个进程之间的通信就像平时发邮件一样，你来一封，我回一封，可以频繁沟通了。

但邮件的通信方式存在不足的地方有两点，

- 一是通信不及时
- 二是附件也有大小限制，这同样也是消息队列通信不足的点。

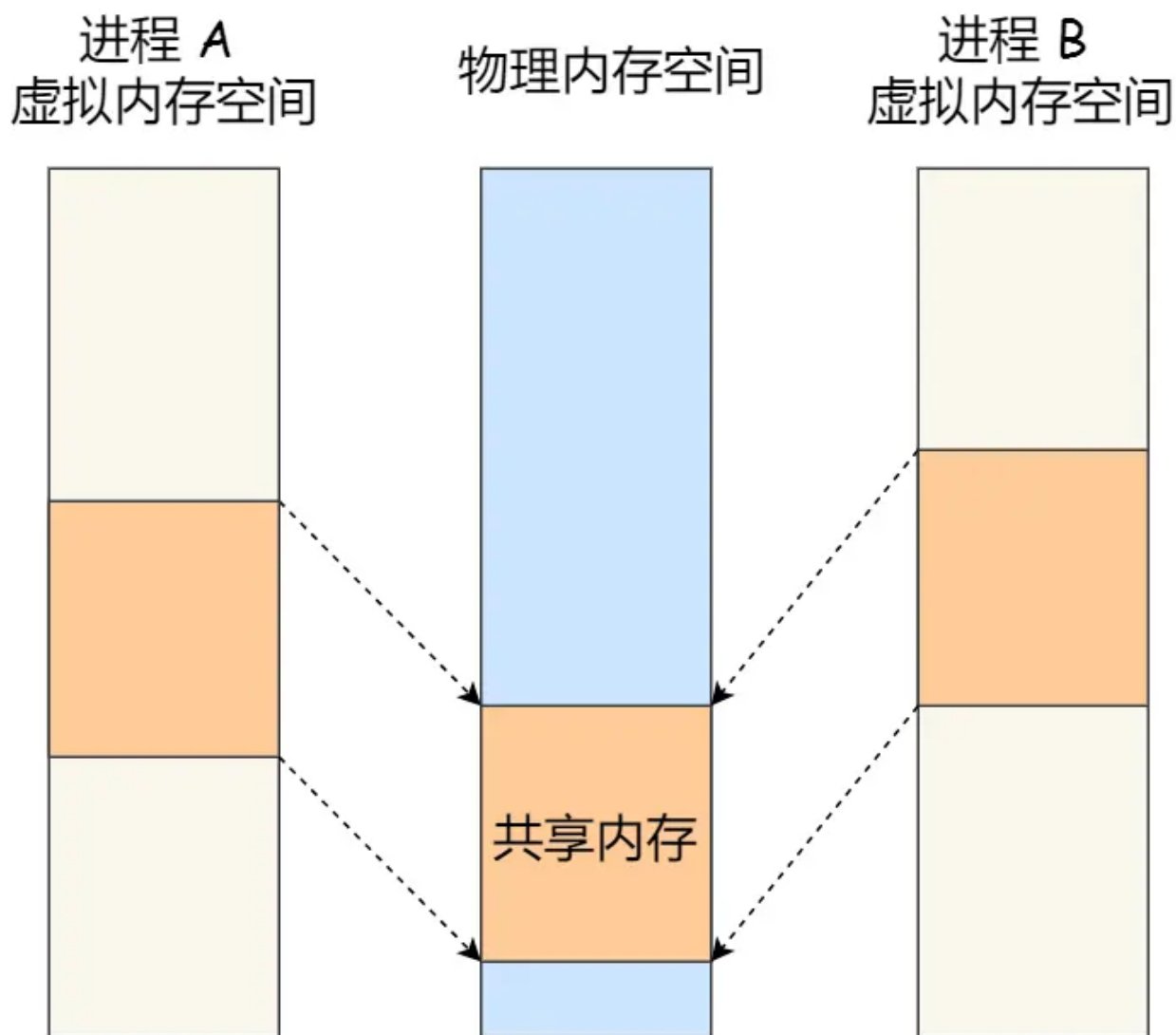
消息队列不适合比较大数据的传输，因为在内核中每个消息体都有一个最大长度的限制，同时所有队列所包含的全部消息体的总长度也是有上限。在 Linux 内核中，会有两个宏定义 MSGMAX 和 MSGMNB，它们以字节为单位，分别定义了一条消息的最大长度和一个队列的最大长度。

消息队列通信过程中，存在用户态与内核态之间的数据拷贝开销，因为进程写入数据到内核中的消息队列时，会发生从用户态拷贝数据到内核态的过程，同理另一进程读取内核中的消息数据时，会发生从内核态拷贝数据到用户态的过程

共享内存

像之前的管道/共享内存，都存在从用户空间到内核空间、内核空间到用户空间的拷贝。那么运用共享内存的进程间通信，就很好地解决了这一问题。

共享内存，就是将多个进程的一段虚拟空间映射到同一块物理内存空间，然后通过向物理内存中写数据、读数据，实现进程间通信



信号量

如果出现多个进程争抢资源的情况，多半就会产生问题，所以这时候就需要一个能是进程间互斥访问资源的技术，确保任何时刻只有一个进程访问共享资源，信号量就可以解决这个问题。

信号量不仅可以实现访问的互斥性，还可以实现进程间的同步，信号量其实是一个计数器，表示的是资源个数，其值可以通过两个原子操作来控制，分别是P操作和V操作。

可以理解为P操作就是是这个计数器-1（资源被占用），类似于上锁的效果；

得等到V操作让计数器+1（资源被释放），类似于解锁效果，其他想要使用该资源的进程才可以使用。

信号

上面说的进程间通信，都是常规状态下的工作模式。对于异常情况下的工作模式，就需要用【信号】的方式来通知进程。

比如在shell终端运行的进程，我们可以通过一些组合案件来给进程发信号，来实现进程的杀死、挂起等操作。

- **ctl+c**发送SIGINT信号，杀死进程
- **ctl+z**发送SIGTSTP信号，表示暂时暂停该进程，挂起

如果在后台运行的进程，还可以通过kill指令，来个对应进程发信号：**kill -信号的序号 进程PID**

信号是异步通信机制，可以在任何时候给进程发信号，一旦进程收到信号，就会有一下几种处理办法：

- 执行默认操作：**Linux** 对每种信号都规定了默认操作，例如，上面列表中的SIGTERM 信号，就是终止进程的意思。
- 捕捉信号：我们可以为一个信号定义一个捕捉处理函数，等到进程收到该信号后，就可以执行相应的处理函数
- 忽略信号：当我们不希望处理某些信号的时候，就可以忽略该信号，不做任何处理。但是有两个信号时无法被捕捉和忽略的：**SIGKILL**和**SIGSTOP**，他们用于在什么时候中断或结束某一进程

Socket

前面提到的所有通信方式，都是在同一台主机上，进行进程间通信，如果想要跨网络与不同主机上的进程之间通信，就需要Socket通信了。

当然，通过socket也可以和同主机的其他进程进行通信。

socket系统调用：

```
int socket(int domain,int type,int protocol);
```

三个参数分别代表：

- **domain:** 用来指定协议族，比如 AF_INET 用于 IPV4、AF_INET6 用于 IPV6、AF_LOCAL/AF_UNIX 用于本机；
- **type:** 用来指定通信特性，比如 SOCK_STREAM 表示的是字节流，对应 TCP、SOCK_DGRAM 表示的是数据报，对应 UDP、SOCK_RAW 表示的是原始套接字；
- **protocol :** 原本是用来指定通信协议的，但现在基本废弃。因为协议已经通过前面

进程调度算法

1、批处理系统中的调度

（1）先来先服务：

非抢占式的调度算法，按照请求的顺序进行调度。

有利于长作业，但不利于短作业，因为短作业必须一直等待前面的长作业执行完毕才能执行，而长作业又需要执行

很长时间，造成了短作业等待时间过长。

（2）最短作业优先：

非抢占式的调度算法，按估计运行时间最短的顺序进行调度。

长作业有可能会饿死，处于一直等待短作业执行完毕的状态。因为如果一直有短作业到来，那么长作业永远得不到

调度。

（3）最短剩余时间优先：

最短作业优先的抢占式版本，按剩余运行时间的顺序进行调度。

当一个新的作业到达时，其整个运行时间与当前进程的剩余时间作比较。如果新的进程需要的时间更少，则挂起当

前进程，运行新的进程。否则新的进程等待。

2、交互式系统中的调度

（1）时间片轮转调度

将所有就绪进程按 FCFS 的原则排成一个队列，每次调度时，把 CPU 时间分配给队首进程，该进程可以执行一个时

间片。当时间片用完时，由计时器发出时钟中断，调度程序便停止该进程的执行，并将它送往就绪队列的末尾，同

时继续把 CPU 时间分配给队首的进程。

（2）优先级调度

为每个进程分配一个优先级，按优先级进行调度。为了防止低优先级的进程永远等不到调度，可以随着时间的推移

增加等待进程的优先级。

（3）多级队列

一个进程需要执行 100 个时间片，如果采用时间片轮转调度算法，那么需要交换 100 次。

多级队列是为这种需要连续执行多个时间片的进程考虑，它设置了多个队列，每个队列时间片大小都不同，例如

1,2,4,8,...。进程在第一个队列没执行完，就会被移到下一个队列。这种方式下，之前的进程只需要交换 7 次。

每个队列优先权也不同，最上面的优先权最高。因此只有上一个队列没有进程在排队，才能调度当前队列上的进

程。

可以将这种调度算法看成是时间片轮转调度算法和优先级调度算法的结合。

（4）最短进程优先

如果我们将每一条命令的执行看作是一个独立的“作业”，则我们可以通过首先运行最短的作业来使响应事件最短

进程同步的四种方法

临界区

对临界资源进行访问的那段代码称为临界区

为了互斥访问临界资源，每个进程在进入临界区之前，需要先进行检查

同步与互斥

- 同步：多个进程因为合作产生的直接制约关系，使得进程有一定的先后执行关系
- 互斥：多个进程在同一时刻只有一个进程能进入临界区

信号量

信号量是一个整型变量，可以对其执行down和up操作，也就是常见的P和V操作。

- down：如果信号量大于0，执行-1操作；如果信号量等于0，进程睡眠，等待信号量大于0
- up：对信号量执行+1操作，唤醒睡眠的进程让其完成down操作

down和up操作需要被设计成原语，不可分割，通常的做法是在执行这些操作的时候屏蔽中断

如果信号量的取值只能为0或1，那么就成为了互斥量（Mutex），0表示临界区已经加锁，1表示临界区解锁。

管程

使用信号量机制实现的生产者消费者问题需要客户端代码做很多控制，而管程把控制的代码独立出来，不仅不容易出错，也使得客户端代码调用更容易

C语言不支持管程，下面的示例代码使用了类Pascal原因呢来描述管程。示例代码的管程提供了insert（）和remove（）方法，客户端代码通过调用者两个方法来解决生产者-消费者问题

管程有一个重要特性：在一个时刻只能有一个进程使用管程。继承在无法继续执行的时候不能一直占用管程，否则其他进程永远不能使用管程

管程引入了条件变量以及相关的操作：**wait（）**和**signal（）**来实现同步操作。对条件变量执行**wait（）**操作会导致调用进程阻塞，把管程让出来给另一个进程持有**signal（）**操作用于唤醒被阻塞的进程

存储系统

页面置换算法

1.最佳页面置换算法（**OPT**）

置换在【未来】最长时间不访问的页面，但是实际系统中无法实现，因为程序访问页面时是动态的，我们无法预知每个页面在【下一次】访问前的等待时间，因此作为实际算法效率衡量标准

2.先进先出置换算法（**FIFO**）

将页面以队列形式保存，先进入队列的页面先被置换进入磁盘

3.最近最久未使用的置换算法（**LRU**）

根据页面未被访问时长用升序列表将页面排列，每次将最久未被使用页面置换出去

4.时钟页面置换算法

把所有的页面都保存在一个类似钟面的【环形链表】中，页面包含一个访问位

当发生缺页中断时，顺时针遍历页面，如果访问位为1，将其改为0，继续遍历，直到访问到访问位为0的页面，进行置换

5.最不常用算法

记录每个页面访问次数，当发生缺页中断时候，将访问次数最少的页面置换出去，此方法需要对每个页面访问次数统计，产生额外开销

分段

虚拟内存采用的是分页技术，也就是将地址空间划分成固定大小的页，每一页再与内存进行映射

如果使用分页系统的一维地址空间，动态增长的特点会导致覆盖问题的出现。

分段的做法是把每个表分成段，一个段构成一个独立的地址空间。

每个段的长度可以不同，并且可以动态增长

1.纯分段

分段和分页本质上是不同的，页面是定长的而段不是。

优点：

- （1）共享：有助于几个进程之间共享过程和数据。比如共享库
- （2）保护：每个段都可以独立地增大或减小而不会影响其他的段

2.分段和分页结合

程序的地址空间划分成多个拥有独立地址空间的段，每个段上的地址空间划分成大小相同的页。

这样即拥有分段系统的共享和保护，又拥有分页系统的虚拟内存功能

3.分段与分页的比较

对程序员的透明性：

分页透明，但是分段需要程序员显示划分每个段

地址空间的维度：

分页是一维地址空间，分段是二维的

大小是否可以改变：

页的大小不可变，段的大小可以动态改变

出现的原因：

分页主要用于实现虚拟内存，从而获得更大的地址空间；分段主要是为了使程序和数据可以被划分为逻辑上独立的地址空间并且有助于共享和保护

进程终止的方式

进程的终止

进程在创建之后，它就开始运行并做完成任务。然而，没有什么事儿是永不停歇的，包括进程也一样。进程早晚会发生终止，但是通常是由于以下情况触发的

正常退出(自愿的)

错误退出(自愿的)

严重错误(非自愿的)被其他进程杀死(非自愿的)

正常退出

多数进程是由于完成了工作而终止。当编译器完成了所给定程序的编译之后，编译器会执行一个系统调用告诉操作系统它完成了工作。这个调用用在 UNIX 中是 **exit**，在 Windows 中是 **ExitProcess**。面向屏幕中的软件也支持

自愿终止操作。字处理软件、Internet 浏览器和类似的程序中总有一个供用户点击的图标或菜单项，用来通知进程删除它锁打开的任何临时文件，然后终止。

错误退出

进程发生终止的第二个原因是发现严重错误，例如，如果用户执行如下命令

为了能够编译 `foo.c` 但是该文件不存在，于是编译器就会发出声明并退出。在给出了错误参数时，面向屏幕的交互式进程通常并不会直接退出，因为这从用户的角度来说并不合理，用户需要知道发生了什么并想要进行重试，所以

这时候应用程序通常会弹出一个对话框告知用户发生了系统错误，是需要重试还是退出。

严重错误

进程终止的第三个原因是由进程引起的错误，通常是由于程序中的错误所导致的。例如，执行了一条非法指令，引用不存在的内存，或者除数是 0 等。在有些系统比如 `UNIX` 中，进程可以通知操作系统，它希望自行处理某种类型的错误，在这类错误中，进程会收到信号（中断），而不是在这类错误出现时直接终止进程。

被其他进程杀死

第四个终止进程的原因是，某个进程执行系统调用告诉操作系统杀死某个进程。在 `UNIX` 中，这个系统调用是 `kill`。在 `Win32` 中对应的函数是 `TerminateProcess`（注意不是系统调用）。

守护进程、僵尸进程和孤儿进程

守护进程

指在后台运行的，没有控制终端与之相连的进程。它独立于控制终端，周期性地执行某种任务。`Linux` 的大多数服务器就是用守护进程的方式实现的，如 `web` 服务器进程 `http` 等

创建守护进程要点：

（1）让程序在后台执行。方法是调用 `fork()` 产生一个子进程，然后使父进程退出。

（2）调用 `setsid()` 创建一个新会话。控制终端、登录会话和进程组通常是从父进程继承下来的，守护进程要摆脱它们，不受它们的影响，方法是调用 `setsid()` 使进程成为一个会话组长。`setsid()` 调用成功后，进程成为新的会话组长和进程组长，并与原来的登录会话、进程组和控制终端脱离。

（3）禁止进程重新打开控制终端。经过以上步骤，进程已经成为一个无终端的会话组长，但是它可以重新申请打开一个终端。为了避免这种情况发生，可以通过使进程不再是会话组长来实现。再一次通过 `fork()` 创建新的子进程，使调用 `fork` 的进程退出。

（4）关闭不再需要的文件描述符。子进程从父进程继承打开的文件描述符。如不关闭，将会浪费系统资源，造成进程所在的文件系统无法卸下以及引起无法预料的错误。首先获得最高文件描述符值，然后用一个循环程序，关闭 0 到最高文件描述符值的所有文件描述符。

(5) 将当前目录更改为根目录。

(6) 子进程从父进程继承的文件创建屏蔽字可能会拒绝某些许可权。为防止这一点，使用 `unmask(0)` 将屏蔽字清零。

(7) 处理 `SIGCHLD` 信号。对于服务器进程，在请求到来时往往生成子进程处理请求。如果子进程等待父进程捕获状态，则子进程将成为僵尸进程 (`zombie`)，从而占用系统资源。如果父进程等待子进程结束，将增加父进程的负担，影响服务器进程的并发性能。在 `Linux` 下可以简单地将 `SIGCHLD` 信号的操作设为 `SIG_IGN`。这样，子进程结束时不会产生僵尸进程。

孤儿进程

如果父进程先退出，子进程还没退出，那么子进程的父进程将变为 `init` 进程。（注：任何一个进程都必须有父进程）。

一个父进程退出，而它的一个或多个子进程还在运行，那么那些子进程将成为孤儿进程。孤儿进程将被 `init` 进程(进程号为1)所收养，并由 `init` 进程对它们完成状态收集工作。

僵尸进程

如果子进程先退出，父进程还没退出，那么子进程必须等到父进程捕获到了子进程的退出状态才真正结束，否则这个时候子进程就成为僵尸进程。

设置僵尸进程的目的是维护子进程的信息，以便父进程在以后某个时候获取。这些信息至少包括进程ID，进程的终止状态，以及该进程使用的CPU时间，所以当终止子进程的父进程调用 `wait` 或 `waitpid` 时就可以得到这些信息。如果一个进程终止，而该进程有子进程处于僵尸状态，那么它的所有僵尸子进程的父进程ID将被重置为1（`init` 进程）。继承这些子进程的 `init` 进程将清理它们（也就是说 `init` 进程将 `wait` 它们，从而去除它们的僵尸状态）。

操作系统在对内存进行管理的时候需要做些什么？

- 操作系统负责内存空间的分配与回收
- 操作系统需要提供某种技术从逻辑上对内存空间进行扩充
- 操作系统需要提供地址转换功能，负责程序的逻辑地址与物理地址的转换
- 操作系统需要提供内存保护功能。保证各进程在各自存储空间内运行，互不干扰

虚拟内存的目的是什么？

虚拟内存的目的是为了让物理内存扩充成更大的逻辑内存，从而让程序获得更多的可用内存

为了更好的管理内存，操作系统将内存抽象成地址空间。每个程序拥有自己的地址空间，这个地址空间被分割成多个块，每一块称为一页。

这些页被映射到物理内存，但不需要映射到连续的物理内存，也不需要所有的页都必须在物理内存中。当程序引用到不再物理内存中的页时，由硬件执行必要的映射，将缺失的部分装入物理内存中并重新执行失败的指令

从上面的描述中可以看出，虚拟内存允许程序不用将地址空间中的每一页都映射到物理内存，也就是说一个程序不需要全部调入内存就可以运行，这使得有限的内存运行大程序成为可能。

例如有一台计算机可以产生16位地址，那么一个程序的地址空间范围是0-64k。该计算机只有32KB的物理内存，虚拟内存技术允许该计算机运行一个64K大小的程序

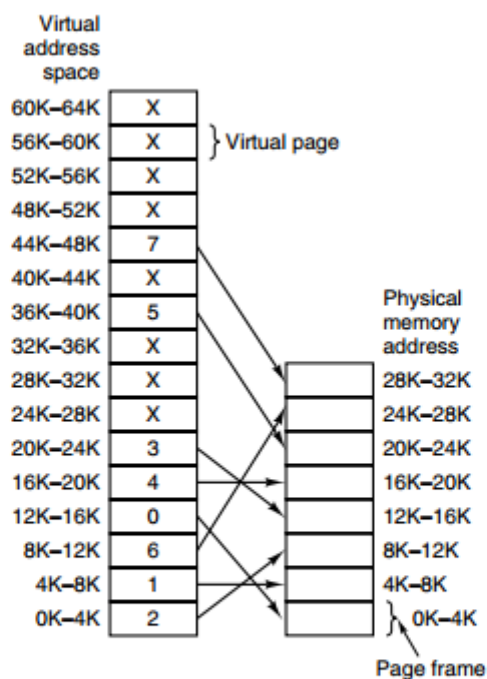


Figure 3-9. The relation between virtual addresses and physical memory addresses is given by the **page table**. Every page begins on a multiple of 4096 and ends 4095 addresses higher, so 4K-8K really means 4096-8191 and 8K to 12K means 8192-12287.

逻辑地址和物理地址

编译时只需确定变量x存放的相对地址是100（也就是说相对于进程在内存中的起始地址而言的地址）

CPU想要找到x在内存中的实际存放位置，只需要用进程的起始地址+100即可。

相对地址又称逻辑地址，绝对地址又称物理地址

锁（lock）

读写锁

- 多个读者可以同时进行读
- 写者必须互斥（只允许一个写着写，也不能读者写者同时进行）
- 写者优先于读者（一旦有写者，则后续读者必须等待，唤醒时优先考虑写者）

互斥锁

一次只能一个线程拥有互斥锁，其他线程只有等待

互斥锁是在抢锁失败的情况下主动放弃CPU进入睡眠状态直到锁的状态改变时再唤醒，而操作系统负责线程调度，为了实现锁的状态发生改变时唤醒阻塞的线程或者进程，需要把锁交给操作系统管理，所以互斥锁在加锁操作时设计上下文的切换。

互斥锁实际的效率还是可以让人接受的，加锁的时间大概100ns左右，而实际上互斥锁的一种可能的实现是先自旋一段时间，当自旋的时间超过阈值之后再将线程投入睡眠中，因此在并发运算中使用互斥锁（每次占用锁的时间很短）的效果可能不亚于使用自旋锁

条件变量

互斥锁一个明显的缺点是他只有两种状态：锁定和非锁定。而条件变量通过允许线程阻塞和等待另一个线程发送信号的方法弥补了互斥锁的不足，他常和互斥锁一起使用，以免出现竞态条件

当条件不满足时，线程往往解开相应的互斥锁并阻塞线程然后等待条件发生变化。一旦其他的某个线程改变了条件变量，他将通知相应的条件变量唤醒一个或多个正被此条件变量阻塞的线程

总的来说互斥锁是线程间互斥的机制，条件变量则是同步机制。

自旋锁

如果进程无法获得锁，进程不会立刻放弃CPU时间片，而是一直循环尝试获取锁，直到获取为止。

如果别的线程长时期占有锁，那么自旋锁就是在浪费CPU做无用功，但是自旋锁一般应用于加锁时间很短的场景，这个时候效率比较高

###

内存

内存覆盖是什么？有什么特点

由于程序运行时并非任何时候都要访问程序及数据的各个部分（尤其是大程序），因此可以把用户空间分成一个固定区和若干个覆盖区

将经常活跃的部分放在固定区，其余部分按照调用关系分段，首先将那些即将要访问的段放入覆盖区，其他段放在外存中，在需要调用前，系统将其调入覆盖区，替换覆盖区中原有的段。

覆盖技术的特点：是打破了必须将一个进程的全部信息装入内存后才能运行的限制，但当同时运行程序的代价量大于主存时仍不能运行，再而，内存中能够更新的地方只有覆盖区的段，不在覆盖区的段会常驻内存。

内存交换是什么？有什么特点

交换（对换）技术的设计思想：内存空间紧张时，系统将内存中某些进程暂时换出外存，把外存中某些已具备运行条件的进程换入内存（进程在内存与磁盘间动态调度）

换入：把准备好竞争CPU运行的程序从辅存移到内存

换出：把处于等待状态（或CPU调度原则下被剥夺运行权力）的程序从内存移到辅存，把内存空间腾出来

什么时候会进行内存的交换？

内存交换通常在许多进程运行且内存吃紧时进行，而系统负荷降低就暂停。

例如：在发现许多进程运行时经常发生缺页，就说明内存紧张，此时可以换出一些进程；如果缺页率明显下降，就可以暂停换出。

终端退出，终端运行的进程会怎样？

终端在退出时会发送SIGHUP给对应的bash进程，bash进程收到这个信号后首先将它发给session下面的进程，如果程序没有对SIGHUP信号做特殊处理，那么进程就会随着终端关闭而退出。

什么是局部性原理，有哪两大局部性原理？

主要分为时间局部性和空间局部性。

时间局部性：如果执行了程序中的某条指令，那么不久后这条指令很有可能再次执行；如果某个数据被访问过，不久后该数据很可能再次被访问。（因为程序中存在大量的循环）

空间局部性：一旦程序访问了某个存储单元，在不久之后，其附近的存储单元也很可能被访问。（因为很多数据在内存中都是连续存放的，并且程序的指令也是顺序地在内存中存放的）

malloc的底层实现

C语言中用malloc来在用户空间中进行动态内存分配。但是作为库函数，其本质就是对系统调用进行了一层封装，因此在不同的系统下实现不同。

在Linux中，当申请的内存小于128K时，malloc会使用brk()函数在堆区分配内存。当申请内存大于128K的空间时，会使用mmap在映射区进行分配。

但是如果每次申请都要大量的CPU开销，而且还会产生内存碎片，就得不偿失了。

如何解决？

为了解决这个问题，malloc在底层采用了内存池来解决这个问题

他会先申请大量内存作为堆区，然后将这块内存拆分为多个不同大小的内存块，以块为基本单位。同时，会使用隐式链表来连接所有的内存块，包括已分配块和未分配块。为了方便内存块的管理，malloc采用显式链表来管理所有的空闲块。

当我们调用 `malloc` 进行内存分配时，就会去搜索空闲链表，找到满足需求的内存块，如果内存块过大，则会将内存块拆分为两部分，即一部分用来分配，另一部分则变为新的空闲块。

当我们释放内存块时，会通过遍历隐式链表，判断释放块前后内存块是否空闲，来决定是否需要合并内存块